

CS261 Final Report

Young-Ha Chun

Owen Green

Archie Harrodine

Tudor Irimies

Yi Xiang Ng

Monday 13th March, 2023

1 Introduction

This final report is the culmination of the work put in by our team for the past term. This report describes the software risk management platform we produced, the significant changes and alterations from our original design, how the system was developed, and an overall evaluation of the project's success. We intend to showcase just how our product is effective at tracking software projects, and calculating their risk of failure.

2 System Overview

The overarching system of the project enables a team of developers to track their progress of a software engineering project, and be made aware of how at risk their project is of failure. Our system consists of three large elements based on the model view controller architecture. These elements are as follows: the website displaying to the user the key interactable features, the intermediary database storing information accessible to both the website and the back end, and the back end risk calculator, which determines the risk of failure for specific projects and provides custom feedback.

The risk calculator was one of the primary focuses of the back end development. By following the encapsulation design principle, the system was developed with the purpose of promoting modularity, code reusability and ease of use. Implemented as a singleton class, it was done with the intention of allowing for not only our development team to work independently, but also enabling a more maintainable and expandable system. The system consists of an object through which a series of functions may be accessed that let us calculate an associated risk value for a given metric, and a given project. Each associated calculation makes use of extensive research, and the acquisition of certain datasets which let us come to more conclusive values. Crucially, the calculator is a cumulative product, which uses an exponential moving average to take into account the whole scope of the project it is evaluating. The metrics that are currently used are requirement severity analysis, schedule analysis, cost analysis and developer assessed quality; however due to its black box design, integration of other metrics are able to be easily addressed and considered, without damaging the integrity of the whole project.

Secondly, the database system is making calls to the risk calculator, obtaining updated values concerning current projects, storing it and providing access to it for the website to display. The system is supported by Amazon Web Services (AWS), in particular the tools of DynamoDB. DynamoDB is a managed proprietary NoSQL database service which enables the storing of a series of tables with the relevant information, without explicit relationships between tables. Despite some drawbacks discussed in a later section, it allowed for easy-to-use data storage that both the controller and the view would be able to interact with. The design mimics relational databases, with a series of tables storing connected, relevant pieces of data that we want to display to the user, along with the added benefit of additional flexibility as a result of NoSQL, allowing for a greater support of user custom requirements and information. Inside the database, projects and their current sprints are tracked, while also storing past sprint data, to provide more information to the risk calculator, as well as demonstrate and visualise trends for the users. It additionally stores details of the employees and managers, thus linking with the User Pool created by Amazon Cognito, which will be discussed later.

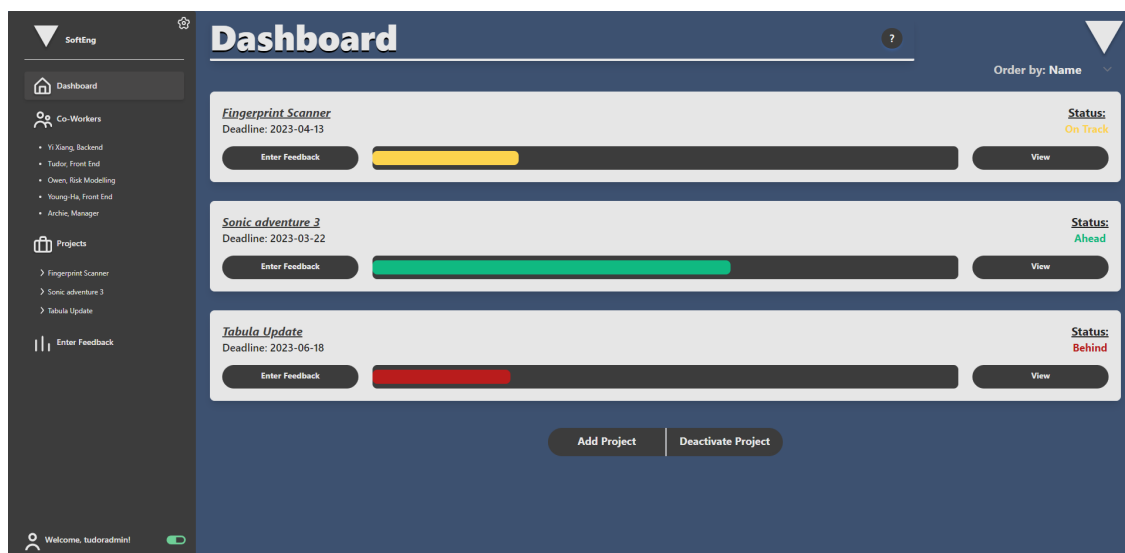


Figure 1: Dashboard Page

The final important aspect of the system is the user interface. The visual display presented to users is a critical aspect of the system as it facilitates their interaction and engagement with the product. The system is first accessed via a login page, organised and managed by Amazon Cognito. The users are prompted to log in using their credentials, after which a session is created that enables identification of the user as they navigate through the pages. The session is remembered once a user logs in, thus eliminating the need to re-enter login details repeatedly until the session expires. This allows users to move seamlessly between tasks without interruption. After successful login, the user is shown the dashboard page, which serves as the home for all encounters on the website. A key feature for new users is an easily accessible tutorial available through the help button located at the top right. The tutorial provides a comprehensive explanation of the page's interactions and features, tailored to the user's role within the company. The website also features a navigation bar on the left-hand side of the page. It is always present on the website, highlighting the page the user is currently on and facilitating movement between pages and sections of the website. From the dashboard page, users can view and order projects according to various metrics, such as by name, budget, or deadline, thus allowing for a more personalised experience, as well create and deactivate new projects, through two separate buttons. For creating, an overlay will appear, prompting the user to input starting information about the project, such as the name, budget, and sprints information, while for deactivating a project, the user is asked to select the appropriate project.

Figure 2: Add Project View

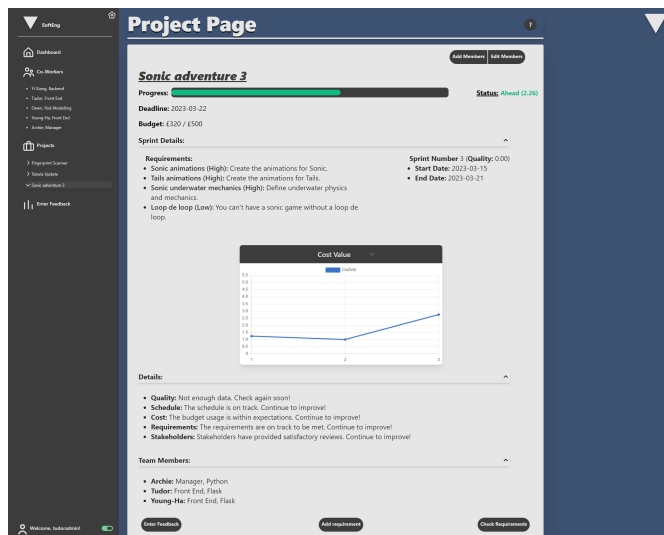


Figure 3: Project Page

The project page can be accessed via the corresponding view button on the dashboard or the navigation bar as previously mentioned. Similarly, the project page also includes a help button that provides explanations on how to use the page. An important feature of the project page is that details are displayed according to the permission level of the user, for a given project, ranging from 1 to 3 with the highest access given to a manager. As such, depending on the user's permission, they have access to different areas of the website. For those with the lowest permission, level 1, the user may view basic project information consisting of the: project name, progress bar, status, deadline and budget. For level 2 users, they gain access to two new sections on the page, composed of the "Sprint Details" where they may view current requirements to be completed with a graph showing a detailed overview of sprints thus far, as well as the "Team Members" section where they may view coworkers in the same project. Level 3 users would also have access to a "Details" component, where they are given feedback on various metrics such as their budget spending and sprint timings, to get a better understanding of how to improve. Additionally, at the highest level of permissions, the manager has access to features that enable them to alter project data in the database, from editing members and their permissions, to adding and completing requirements.

All aforementioned sections can be collapsed via an accordion to minimise the space taken on the screen, to aid with potential visual clutter. Lastly, the feedback page provides an interface for the user to contribute to the progress tracking by inputting a number of metrics to aid in determining the status of a project. The page is able to take the user inputs which will then be factored into the risk modelling system, while the user is redirected to the dashboard page.

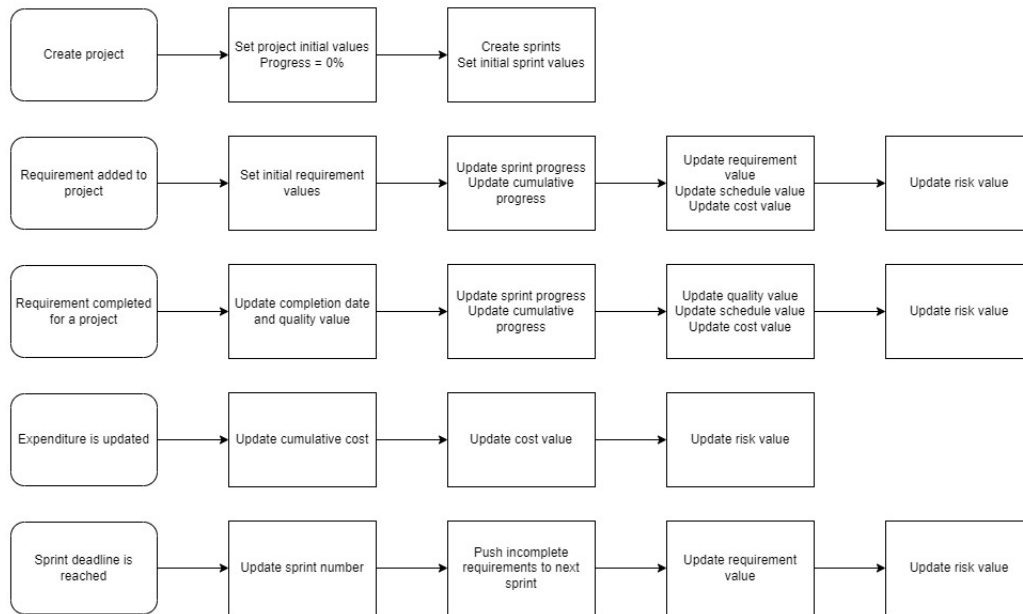


Figure 4: Component Interaction Diagram

In a more general view, each page of the website communicates together through the back end, calling a collection of functions which interact with the database and the risk calculator. The implementation considers main events which can cause a change, that are mapped out with the relevant updates needed to be made to the database. The interaction between the three components flows easily, as attribute updates occur only when triggered by an event. As the risk value is affected by a combination of most events, it is the ending point for most interactions. When the database is updated, the risk calculator is called to gather and update the required data.

3 System Changes

3 i Requirement Changes

Below is an overview of the requirements that were either met or unchanged, and the impact behind it. Requirements that have been met are not specified, as they are demonstrated through the system overview, and throughout the remainder of the document.

Changed requirements

Requirement	Change	Impact
CR1.5, DR1.5	Tutorial mode has been changed with a help button on each page.	Allows for less data tracking, and an easier way to always remind yourself how to use the website. Was significantly easier to implement as well.
DR1.8	Instead of setting customized permissions to team members, there are three levels depending on the role within the company.	Reduces the complexity of the project, however it meant that the resourcing metric was unable to be met.
DR1.9	Opposed to accommodating both plan-driven and AGILE approaches, we focused in on only AGILE.	This reduced the complexity, and made the focus on the much more widely used type of methodology, but does alienate projects using plan-driven approaches.
CR2.1	In place of a temperature bar, we have used a lexical approach, alongside displaying numerical information in the form of a graph.	It may be slightly harder to interpret the data for an average viewer, but made its design far easier.
CR2.2	Qualitative parameters were mostly excluded.	This means that the data is easier to interpret, but some information from the team, and their well-being is lost.
CR2.3	Instead of directly controlling which measurements can be entered, we instead limit how the user can interact with the project.	This was more an indirect result of certain metrics not being implemented.
DR2.1	Change risk value range from $[0,1]$ to $[0,5]$.	An indirect result of it working out nicer to keep a consistency amongst all metric evaluations.

Unmet requirements

Requirement	Why it was not achieved	Impact
CR1.8	The inclusion of presets was low priority, and added more complication to the frontend development.	The setting of team member permissions is slightly more tedious and clunky.
DR1.10	The integration of Github or Jira was one that was not thoroughly explored, due to low priority, and how it was not directly needed for the initial design.	This means some quality of life features of being able to use external tools are lost.
DR2.2	The exclusion of qualitative parameters was done due to a lack of existing research, business precedence, and considerations needed for storing.	This results in neglect of certain softer metrics when considering the status of a project.
DR2.4	Due to time constraints, we were unable to display an ordered list of how the metrics impacted the project.	Project managers will be able to see which metric they needed to most focus on, however not as easily as they will need to read through each metric.
CR4.2	Suggesting new risk metrics was simply out of scope, since we were not able to cover the base selection.	Little change is made, only a loss of some functionality that in future may be implemented.

3 ii Design Changes**Tutorial to help button**

In the Requirements and Analysis document we mentioned a tutorial that users would complete upon their first login of the website listed as requirement DR1.5. However, during the development process we realised that having an attribute in the user table dedicated to a single-time event would not be ideal and would also be counter intuitive if the user would want to go through the tutorial more than once. Therefore, a decision was made to change the tutorial to a “helper” button on each page that can be pressed to display annotated graphics of the page’s layout and a description of each available feature.

Coworkers - no online/offline activity tracker

The initial proposal for the coworkers list in the navigation bar, described in the Planning and Design document, was to also display other members currently using the website through an indicator that would appear green if a coworker was “online” and grey if they were not. A conclusion was made early on that this feature was non-essential and should not be implemented as it would add additional complications to the development of the website, which would impact the progress of essential features.

Database - change of tables/schema

A large amount of changes to the database design mainly stemmed from the realisation that our machine learning program needed more information to properly produce results. This meant more tables such as the Requirement and Sprint Tables, as well as more attributes to existing tables to store more information (such as active status, risk, quality values, etc). Storing these data as well as the sprints definitely made our product better as we now had the option to display to users data in the form of graphs to show overall trends in the project’s development. As for other changes, it mainly came from the frontend team suggesting modifications to the database schema and methods for quality of life adjustments, such as changing data types for attributes and changing getter and setter methods.

Updated Design Direction

One of the largest overall system changes took place early on within actual development. The original design, while it considered the metrics that we may want to take into account, and addresses the various requirements and needs of the project, had a description which was more vague and less specific than it was needed to be. Hence, development was shifted in a new direction, which put far more focus upon the AGILE interpretation. Originally, we were going to allow for the user to choose between both AGILE and plan-driven methodologies, and suit the website appropriate to both (DR1.9). However, very quickly we found this not only to be an unrealistic goal to attain, due to our time and team constraints, but also due to available data. Not only do the vast majority of modern companies use AGILE scrum focused methodologies, due to the modern necessities of customer interaction, but as a result of this it has made most research on how software projects perform, be exclusively investigating AGILE approaches. Therefore, we scrapped the plan-driven side, and only looked at AGILE.

With this shift, we now had the problem of how we were going to store, interpret and show our data. The choice in the approach that was agreed upon was a cumulative sprint analysis. This method was the idea that, at the start of the project, a series of equally lengthed sprints would be determined; from where three sources of data would feed into our risk calculator, and determine the overall risk that the project may fail.

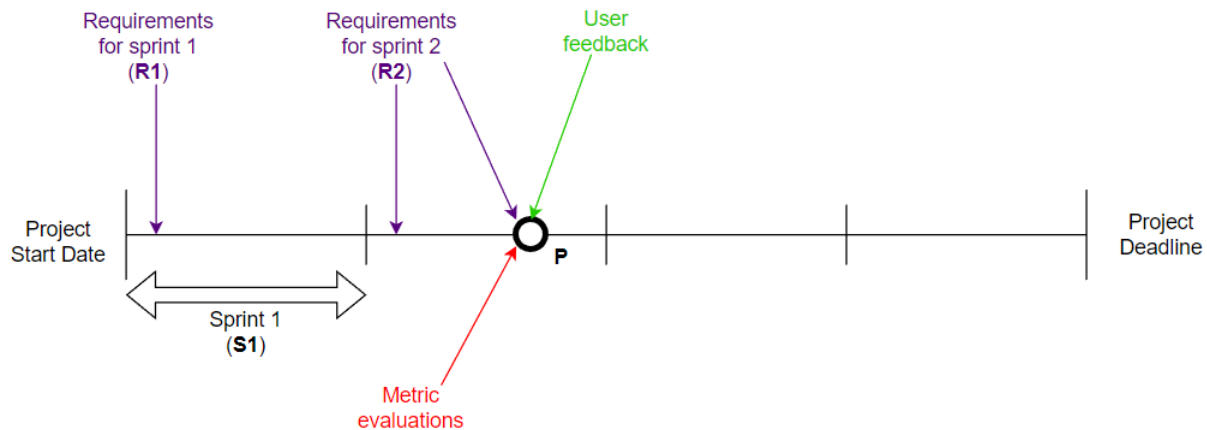


Figure 5: Risk Analysis Visualisation

This diagram showcases how we were visualising the problem. If we suppose the bar is the timeline of the project, at the start of each sprint, signified by each line, a series of requirements (or ‘tasks’) would be set for the team to complete. This diagram has 4 sprints, but there could be any positive number of sprints. During the time of a sprint, information of the metric evaluations, the evaluation of the requirements, and the user (team member) feedback, would all be used to determine an overall value that informs the user of how the sprint is going. In the visualisation above, we use everything up until point P . To evaluate the risk value at a point in Sprint i (S_i) we use the metric evaluation of $S_1, S_2, \dots, S_i$, the requirement evaluation of

requirements for sprint i (R_i) and the user feedback of $S_1, S_2, \dots, S_i$. This all together builds a cumulative image of what we are trying to achieve, and the operations that must be needed to achieve it.

Additionally, we have to consider incomplete requirements, and what happens with them. This new design made the assumption that if a requirement is incomplete, then it must be completed in the next sprint. It is pushed on to the next sprint, and assigned a higher priority to complete.

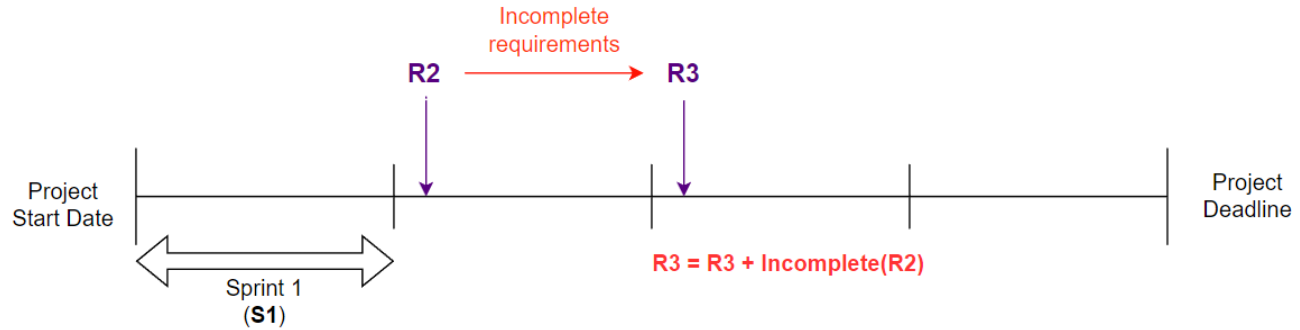


Figure 6: Incomplete Requirement Analysis

The new reworked design once conferred and organised with the team, enabled a focused approach to the work, as more specific tasks had been set, and therefore individual teams of people could work on their side of the project. The tasks consisted of:

1. Collecting the metrics and requirements
2. Storing the metrics and requirements
3. Interpreting the metrics and requirements

Our idea of what metrics we wanted to use and consider, had not changed since our original design, however there was a new need to order and prioritise, as very quickly into the second sprint, the scrum master identified that there was not enough time to achieve all of them. Hence out of the metrics we planned, we prioritised as such:

1. Schedule: exploring the number of requirements met for their deadline.
2. Cost: looking at budget and expenditure.
3. Redundancy scale: exploring the manager-made risk assessment, and realised risk cost.
4. Developer quality: the internal team rating of quality of a requirement.
5. Resourcing: rating the assignment of resources (people) to requirements.
6. Stakeholder satisfaction: the review and evaluation with the client.
7. Business Change: the success of a restructuring of a business.
8. Technical maturity: how mature a system has become.
9. Soft metrics: evaluating the well-being of the team members.

There were several factors at play for which metric we wanted to complete first, in particular: importance, availability of research, and the ease of obtaining data. Schedule and cost metrics were both guaranteed considerations, purely from the perspective of the data being built into the tracking aspect, and they are the two most obvious ones to consider from a practical business perspective. Next the redundancy scale (rename of risk assessment to avoid confusion) is a high priority implementation as according to several sources, including talks with people at larger software companies, the greatest cause of software project failure is failure of risk management, and not accounting enough for it. Next, is developer quality, and this is primarily because it is blocking resourcing, which is another vital metric, which would provide the most concrete areas for the manager to change and improve the project. Stakeholder satisfaction is low here, mostly due to the higher amounts of effort it would take the

web developers to account for; instead we are making the slight assumption that the manager will be taking the customer into account when they decide upon the requirements of the next sprint. So if something is not up to standard with the customer, the next sprint will see a requirement added which tackles that area. The final two hard metrics are business change and technical maturity. These are low down to consider due to the lack of research that we found on them, and from how specific they were, feeling less significant to the remainder of the metrics. This leaves us with the soft metrics. These metrics are the one that regard less tangible concepts, surrounding the well-being of employees, their morale, and the overall productivity of the team. Upon discussion, while these are important and valid metrics to track, due to the lack of this being studied, combined with most real world businesses neglecting these statistics, we decided that we were going to put our time toward the hard metrics. Furthermore, collecting and interpreting soft data is substantially more difficult than tracking physical data like expenditure or progress.

This refocus, and change from the initial design, which was flawed in its vagueness surrounding the specifics, was a necessary change to make. Not only for the final product, but for the effectiveness of our methodology, as now each sprint the implementation toward each next metric could be focused upon. This was a large shift in the design though, and this shift bled out into the subsystems of the project.

4 System Development

4 i Front End

Development Overview

APIs and Services Used

Jinja2 is a website templating engine for Python which is essential in passing the correct data from the database and displaying it on the website. Jinja2 operates by receiving variables from Flask routes and rendering the HTML file (via the `render_template()` function). The values that are passed replace variables/expressions within the template upon render, which are used in cases such as: displaying the username in the navigation bar or the project name on the project page. Furthermore, tags allow us to control the logic of the templates and give access to statements and loops through a Python inspired syntax. These are utilised for restricting particular features, e.g. using if statements to check permission levels, or iterating over a list of values retrieved from a table, for example: displaying the data for each project on the dashboard. Jinja2 also adds an additional layer of security through the automatic escaping of data for HTML files when configured with Flask, which aids in the prevention of Cross Site Scripting (XSS) attacks.

To design the appearance of the website we utilised Tailwind CSS, which is an open source CSS framework. The framework gives access to a large variety of utility classes such as: sizing, colours, shadows etc. that can be added into the class attribute of HTML elements to change their appearance. It operates by identifying the class names in templates and generating a static CSS file that is referenced in the head element of the HTML files. By using Tailwind we were able to produce a user friendly and aesthetically pleasing website, in a more time efficient and easily understandable process than writing custom CSS files.

When we realised that we would need to add collapsable details within the project page, we understood that it would require JavaScript to operate. Although Tailwind had been a great tool for styling the website appearance, it was not an all-encompassing package and lacked some advanced JavaScript implementations. Through researching methods we found an option that would be simpler than manually writing the feature in JavaScript. Tailwind has a large collection of extensions, with some specialising in JavaScript components that can be integrated directly into the HTML files. We had two primary candidates for extensions: DaisyUI and Flowbite. Initially, we tried both and came to the conclusion that using DaisyUI would be advantageous, as the JavaScript implementation seemed to be simpler to use and integrate. However, after further testing we realised that Flowbite worked more seamlessly with the existing Tailwind framework and ultimately would be a better choice to continue with. Beyond the collapsable details in the project page, we also utilised Flowbite for other features for example: the modals (overlays) for the add/deactivate project buttons.

Once we understood that we would need graphs to display the progress of the sprints for each value (cost, schedule etc.), we checked whether Flowbite had any chart integration included in the free package. As we were unable to find any such features, we researched other extensions that had graph implementations readily available and that wouldn't interfere with our current system using Tailwind and Flowbite. We found an extension that fit those criteria called Tailwind Elements. The new extension was able to add Chart.js graphs to the project page with minimal issues and was also used in the "helper" button to display the overlay and the carousel for the annotated images.

Due to the agile methodology used, we were not certain on adding graphs to the sprint details until the final stages of the website's development. If we had planned the inclusion of graphs at an earlier stage then we would have chosen Tailwind Elements as our sole extension, as it included all the components that were required from Flowbite, as well as the ability to apply animations to them, in addition to the graph display. While the current system of using both Flowbite and Tailwind Elements is not an issue, in hindsight using only a single extension would have simplified both the implementation of the features and the code.

By simplifying the JavaScript integration through Flowbite and Tailwind Elements, we were able to reduce the number of potential issues that may have arisen through manual coding. In general, the use of the Tailwind framework and its extensions, made the front end development efficient while not sacrificing the quality of the website's design.

Permissions

As part of the requirements CR1.2 and CR1.6, the user must have access to information about projects appropriate to their role within the company. As such, permissions are an essential aspect for this project management tool, as they allow for the secure management of user access and control over the project. In our website, we have implemented three levels of permission, in increasing order - Outsider/Viewer, Worker/Team Member, and Manager/Leader - to ensure that users can access the necessary information and functionalities according to their role and responsibilities. Level 1 users, also known as Outsiders/Viewers, have access to general project information, such as the project name, progress, deadline, and budget, but not development requirements or detailed progress tracking. Level 2 users, or Workers/Team Members, have access to the same information as Level 1, as well as specific requirements for the current sprint and information about other members working on the project. Level 3 users, or Managers/Leaders, have access to everything that Level 2 users can see, as well as detailed summaries of the project's sprints, the ability to add and check off requirements for the team to work on, and the ability to add new team members. The relationship between the levels of permissions is hierarchical, with Level 1 users having the least amount of access and Level 3 users having the most. This system allows for a streamlined workflow and clear communication between team members, ensuring that everyone has access to the information they need to complete their tasks efficiently.

4 ii Database

From among all available database options provided by Amazon services, we have decided to use DynamoDB, a serverless NoSQL system. The benefits of using DynamoDB are vast, ranging from millisecond performance, easy scalability and ease to integrate with other Amazon Services. Using AWS' Software Development Kit for Python3 (Boto3), we can call add, delete, query and update operations on the Database through a python file. By using AWS, a single user can create roles and permissions for themselves, which in turn will allow users to generate access keys and secret keys which gives us access to the Database in the AWS system.

By making use of Object Oriented Programming, the database team only needed to create methods for accessing, adding and updating to the database, while the frontend and back end teams would only need to import the database program to use the functions. A NoSQL Database, also known as a schema-less database means that all records (called 'Items' in DynamoDB) are essentially Key-Value pairs, which means that other than the primary key, the other values entered into the database do not necessarily have to conform to a schema. Taking advantage of this, we can initiate projects with certain attributes empty to signify that they are incomplete/require someone to check off the tasks.

One of the requirements regarding the database was the ability to give employees permission levels which grants them different clearances to view metric data (DR1.2). We had many ideas on how to implement this, including binary flags and boolean values for permissions, but decided with simple clearance levels, with values 1 to 3, each representing a different number of metrics each team member can view. This is stored in the Project_Team table, which represents a team of people tasked with a project, their manager, and their permissions. Adding on to that, requirement DR1.6 entails that the database will provide additional information about a status of a project when requested. Visually, this will be seen as a progress bar on the main project page. The database will store all the requirements needed for a project and the percentage of requirements completed can be calculated through queries in DynamoDB which will result in the status bar mentioned earlier.

One of the major challenges faced by the team was ironically caused by the supposed advantages of a NoSQL Database. The no-relational style of NoSQL made it difficult to ensure referential integrity. This became an issue in the middle of development when more tables were created, many of which referenced either ProjectID or RequirementID. Additionally, NoSQL also meant the absence of a SERIAL datatype, which proved difficult to automatically generate IDs for us. With the team being more comfortable with SQL databases, this sudden shift did cause some problems.

Initially in our design document we had an original basic outline for an entity relationship diagram. This was designed without knowledge of the NoSQL state of DynamoDB. The design was loose and vague, especially how the metrics related with the project; with the new design direction regarding a focus on sprints and AGILE, the database had to be changed. The new design can be seen in Figure 8.

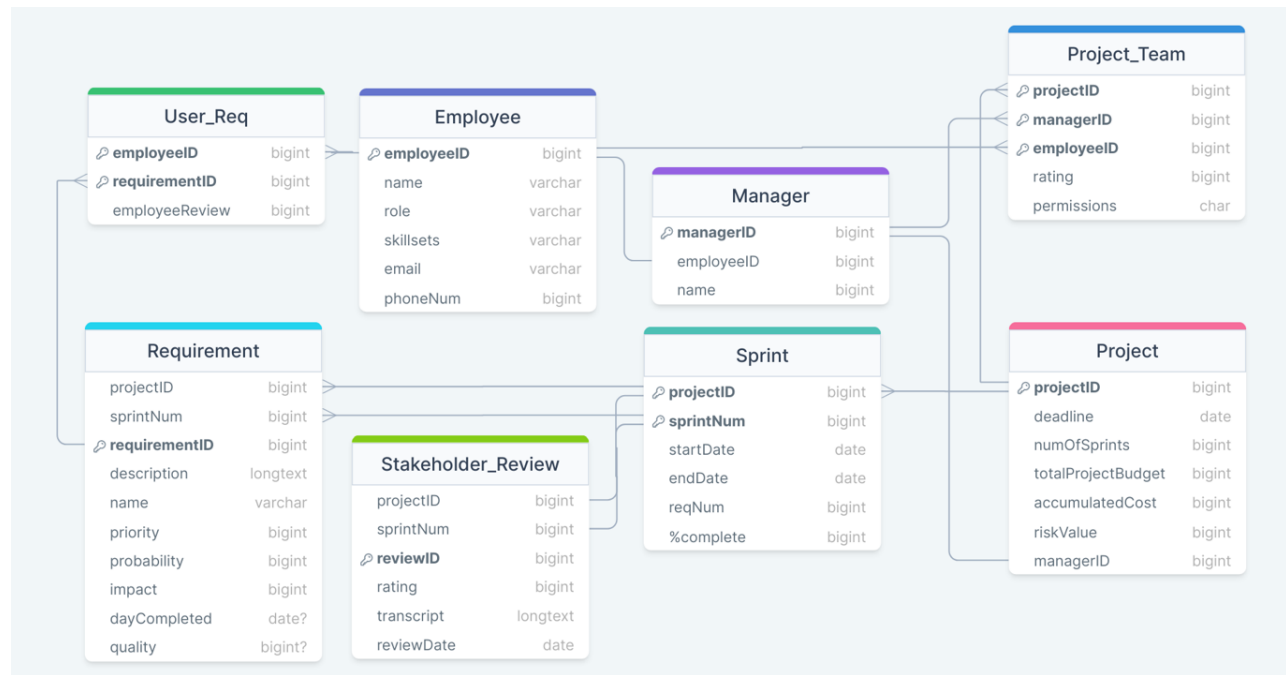


Figure 7: The initial SQL based design.

This design was ambitiously built to reflect what the original final goal of what was set out to accomplish was to look like. The database houses a series of tables, where each project has a set of sprints identified by a composite key of the projectID and sprint number; where each sprint has a series of requirements, associated via a foreign key. This is where details of the project can subsequently be broken down into how they affect each part of the final product.

- The Project table contains overarching details like progress and risk value to display on the dashboard; as well as containing values pertaining to budgeting.
- The Sprints table stored information relating heavily to completion rates, to be used for the schedule metric.
- The Requirements table stored data on what requirements had to be completed, which the requirement analysis could use; and it stored ratings of quality.
- The User_Req table was to pair up team members and requirements, to be used for the resourcing metric.
- The Stakeholder_Review table was for the customer's reviews to be stored and tracked overtime, for the stakeholder metric.
- Employee and Manager tables were to store basic user information to display who is in the team and what skills they possess.
- The Project_Team table was meant for assigning value to how well a team member is performing a project (resourcing) and what access levels they had.

This design was fairly strong. The problem was that it was not well communicated. Due to communication issues further discussed in evaluation, this design was built around the database possessing a relational SQL design. However, when it became apparent this was not the case, combined with delays and timing reducing the number of metrics that could be tackled, in the end the table shifted once more.

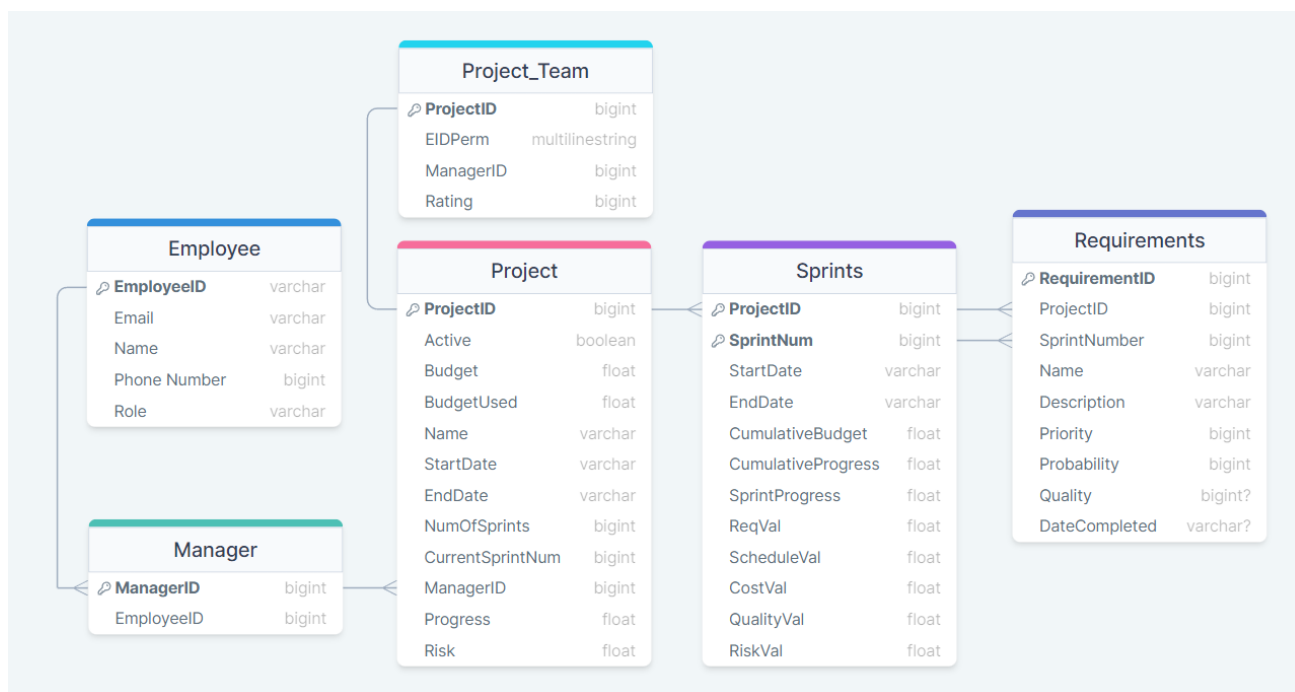


Figure 8: Final Table Structure Used

The table depicted above is the final one that was used, and as we can see it took out some of the original relational complexity. For the most part, the compositional structure remained the same, though the foreign relationships were not strictly enforced by DynamoDB, rather implemented through our code. Additionally, to account for the SERIAL data type problem, we found the best method to account for this to be when we are creating a new entry we access the length of the table, and use this to assign to the new value. This posed risk of what occurs when projects are deleted, but this was dealt with by adding an additional attribute which determined if a project was active or inactive. This is justified by the fact companies may wish to retain failed project data, for future insights.

The next issue that we faced was implementing permissions. This is where a clash of design took place, as while in the original SQL design it was such that in Project_Team, we could remove a many to many relationship and depict it as such. However, during development instead within Project_Team, it was changed so that permissions were reflected by a map in EIDPerm. This map was highly specialised to being a NoSQL feature. Instead of now a single entry reflecting the permissions of an employee in a project, now the table reflected a map for each project of what permissions each member possessed. This indirectly caused the table Project_Team to be a separate table from Project, when it did not now have to be. However, by the time it was addressed, it was built too significantly into our system to quickly adjust it. However, while it does complicate the design, it does still allow for permissions to work.

The final database also saw the removal of several tables and attributes of metrics that were no longer to be considered from time constraints. In particular the stakeholder review, and the resourcing details. Furthermore, we found for tracking the progression of data, as well as for making full use of exponential running average, we would store in the Sprints table, all metrics, so their progression can be detailed.

4 iii Back End

Risk Algorithm

The risk algorithm comprises of 4 different components which are evaluated individually and then combined in order to produce a singular risk value which accurately represents the health of a project. This modular approach also enables easy identification of factors which are most greatly affecting risk (DR2.4). For clarity all risk components were mapped to the interval $[0, 5]$ where 0 represents good, no risk, and 5 represents bad, the highest possible amount of risk. We now consider each of the components in turn and provide an insight into how it is calculated.

Requirement Value - As stated in our planning document we utilised a dataset proposed by Shaukat et al [12] in order to evaluate the risk associated with each requirement. We used a Naïve Bayes classifier, similarly to Akumba et al [3], in order to accurately classify the risk value associated with each requirement. Specifically we used the Gaussian Naïve Bayes function of Scikit-Learn [11] in order to classify requirements. We chose to consider only the priority and probability attributes of the dataset as interpreting the text fields would have increased the complexity of the model greatly and we felt they encapsulated what we needed to measure risk (See **Section 5 iii** for a detailed evaluation of this decision). Once we were familiar with the basic operation of Scikit-Learn the implementation was fairly straightforward leveraging the `test_train_split()`, `GaussianNB.fit()` and `GaussianNB.predict()` functions.

Schedule Value - We analysed a data set of many sprints from various open source projects [7] and found that on average 41.8% of tasks were completed during a sprint. Given the large size of sample data used we assumed that it was normally distributed (Central Limit Theorem [1]) and mapped the probability of a project having a particular completion percentage to the 0 to 5 interval. For example a sprint having a completion rate of 90%, well beyond the normal mean, has a risk value of around 0.35 representing that this factor is contributing very little to the risk of the project. The analysis of the data was performed using pandas [10] as the DataFrame object provided an easy way to filter and concatenate data. For creating and applying the distribution we used `scipy.stats` [13] which with its built in norm and cdf functions made implementation straightforward.

In order to see a picture over multiple sprints we took an exponentially weighted moving average (EWMA) which is shown in the formula below where t represents the number of time steps and each x_i are our input values. A decay parameter, α is used in order to decrease the value of older sprints and hence, consider the most recent ones more prominently. We are able to use this in order to observe the trend of how tasks are being completed across sprints. We implemented EWMA using the built in pandas function and set $\alpha = 0.5$ for a balanced rate of decrease.

$$y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2x_{t-2} + \dots + (1 - \alpha)^tx_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}$$

Quality Value - We use the same ideas outlined in Schedule Value as the `quality_metric` function takes in a list of average quality ratings for each sprint which we can then take the EWMA in order to build a picture across multiple sprints. Implementation was very easy as it uses identical techniques to the schedule value code.

Cost Value - Earned value management is an active research area with many complex techniques being employed to enhance the accuracy of project performance rating [2, 9]. We opted for a more perspicuous approach centering our analysis around the cost performance index (CPI) of a project which is calculated by taking the earned value and dividing it by the actual cost. Projects with an earned value > 1 are under budget and as such are less at risk meanwhile projects with a $CPI < 1$ are over budget. A quick internet search shows there are many different ways of interpreting this figure and acceptable ranges can vary greatly [5]. We opted for a more rigorous interpretation as set out by Kim and Pinto in their article “What CPI = 0.85 Really Means: A Probabilistic Extension of the Estimate at Completion” [6]. They state that typical thresholds for budget are within $\pm 10 - 15\%$ which corresponding to a CPI of over 0.85. However, McKinsey found that large software projects (a budget $\geq \$15$ million) are on average 45% over budget [4]. Therefore, we must accept a wider range of CPI values in order to meaningfully track a wide range of software projects.

For implementation, we created a custom piecewise linear function which maps CPI values onto our 0 to 5 scale taking into account the research of Kim and Pinto as well as the fact software projects are highly likely to run over budget. For CPI over 1.1 we map to 0 risk value as these projects are well within budget. From there we increase gradually so that a project with a CPI of 1 has a risk of 1. We determined that a CPI of 0.5 was a suitable cut off point to maximise the contribution to the overall risk factor of the cost value as this was well beyond McKinsey’s average range which allows our system to have better granularity in determining the cost value.

Due to the linear nature of this function implementation was very straightforward with the linear equations being determined beforehand using Desmos as shown in Figure 9.

Once our 4 risk values have been calculated we can determine an overall risk value by taking a weighted average. This is simply done in the `risk_metric` function which is called by the database helper functions in order to calculate a new risk value for a project. From the above we can see that DR2.1 has clearly been met as our algorithm produces a single values which accurately represents the risk value associated with a project. As DR2.3 is partially met as we can take into account the quality rating provided by team members however, the exclusion of soft metrics, which was done due to time constraints means that DR2.2 and indeed DR2.3 cannot be fully met. The `risk_factor_analysis` function produces a list of the four risk values ordered by priority meaning that DR2.4 has been met. Our modular approach to risk analysis also means that requirements DR4.2 and DR4.3 have been met as new functions can easily be added to `risk_calculator.py`, with only minor adjustments to the `risk_metric` and `risk_factor_analysis` functions.

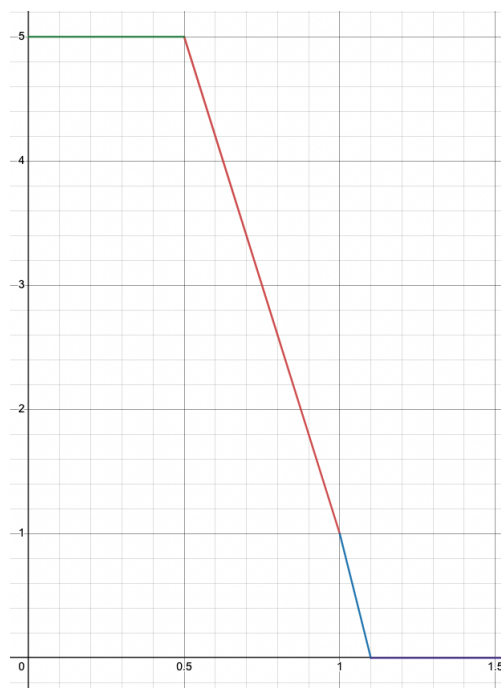


Figure 9: Cost Performance Index Mapping Function

APIs and Services

The backend of the website has been designed using the Python Flask framework for the page routing and Amazon Cognito for the login system. These two tools have been used to create a robust, scalable, and secure website that provides a seamless, user-friendly experience, as required by CR1.1 and CR1.9. Amazon Cognito is a cloud-based authentication, authorisation, and user management service that provides a scalable and secure user directory, named a User Pool, for web and mobile applications, thus creating a secure login system with user authentication, multi-factor authentication, and role-based access control, as specified by CR1.3 and CR1.4. In the context of this project, the Python Flask web framework was used to implement the backend routing of the website. To enable user authentication, the Flask-Cognito-Auth library was utilised, which provides a wrapper around the Amazon Cognito API, enabling the website to interface with the User Pool. This allowed for secure authentication and authorisation of users, with Flask-Cognito-Auth handling the communication between the website and Cognito. More precisely, this library provides decorators that can be used to protect routes in the application, ensuring that only authenticated users can access them.

In most web applications, sessions are used to keep track of a user's interaction with the website. When a user logs in, a unique session ID is generated, and this ID is used to identify the user as they navigate through the site. In the case of our website, sessions are used to authenticate users and retrieve information from the DynamoDB database. When a user logs in, Flask-Cognito-Auth receives an ID token from Cognito, which contains information about the user, such as their email, user ID, and access tokens, which are stored in a user session for later use. This ID token is then used to authenticate the user for subsequent requests. Additionally, the library provides functions to retrieve the user's authentication status, including their username and email, and to logout the user. Once the server has authenticated the user's session, it can then use the session ID to retrieve information from the DynamoDB database. For example, when a user visits their dashboard or project page, the server queries the database for the relevant information and returns it to the user. The server can also update the database with new information based on the user's actions, such as creating or editing a task. The diagram on the right is an accurate representation of the back-end authorisation process when a user attempts to log into the website, where both the AZN and ANS are managed and represented by Amazon Cognito.

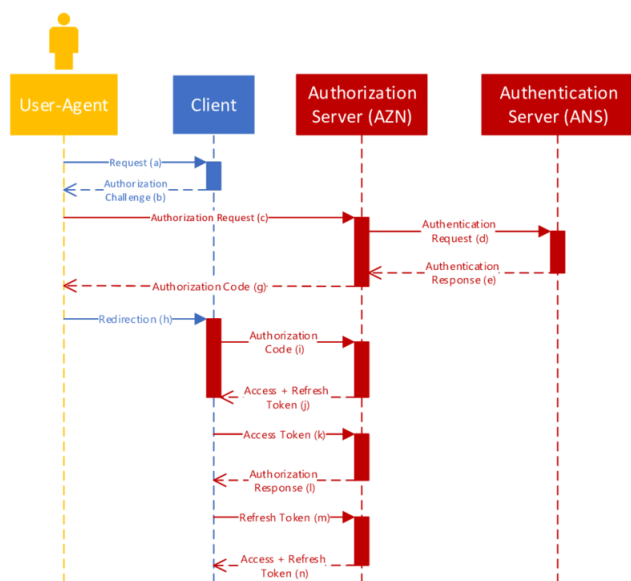


Figure 10: Authorisation Process [8]

4 iv Component interaction

The final aspect to consider during development is the interactivity between the systems. This focus started in the 5th sprint, and had the goal of providing functions that the web page could call to interact with the database. This made use of the interaction diagram 4 to highlight the functions that would need to be generated to work with the website. The interactions were designed this way to minimise the number of unnecessary recalculations and updates, as that is costly on time. A problem that was encountered early on was updating individual components, as up until now there was either the option to update everything, or nothing at all. However, after studying AWS requests, a function was designed such that the tables could update an attribute specified in the function, which reduced heavy amounts of code refactoring. This function was designed for the Project, Sprints and Requirements table, and once tested, made the implementation of the interactions much easier.

The only issue that pertained was feeding the risk calculator singleton the correct data. This posed a slight challenge due to the varying ways that the mathematical aspect could be determined. Some methods within the risk calculator were quite easy, such as requirements only needed a set of pairs of priority and probability; but with the schedule metric it needed an array of sprint progress. How sprint progress is determined is partially up to debate, depending on how the pushing of incomplete requirements to later sprints is tackled. The method that was settled for was to take all complete requirements of the current sprint, and all incomplete requirements of the entire project, and divide the two. This way it makes the natural assumption that if a requirement is incomplete, it is considered in the current sprint. To conclude, the interactivity between components was fairly easy to implement with little issue, as it was carefully planned out, and did not need to perform any complex functionality.

5 Product Evaluation

5 i Front End

Feature Evaluation

We were unable to meet requirement CR2.2, as the risk modelling system was unable to take into account qualitative data. Therefore we could not implement meaningful methods of taking in user feedback, excluding the budget used, which is the only metric that can be inputted in the feedback page and taken into account by the risk modelling system. A solution would require first for the risk modelling to allow for qualitative measurements to be processed, while the front end would have to adjust the feedback inputs to specified metrics and then send the data to the database via a form in a similar manner to the budget. As this would require all three components of the system to be changed, it would be a significant commitment to make.

Similarly, as the number of metrics that could be inputted in the feedback page were limited, we could not implement CR2.3 to allow a manager to select which feedback metrics each user could input. To meet this requirement we would require different levels of feedback input access, akin to the permission level, and therefore require an adjustment of the database as well as an additional feature on the website. This would be added to the edit members overlay along with the edit permissions feature.

While our system for permissions has a hierarchy of levels, there are no options for defaults or presets as mentioned in CR1.8. This requirement would require a change in the database to store permission presets with an option to select a preset to be implemented into the add/edit members overlays.

Although the system relies on Jinja2 to sanitise user input by auto escaping characters, we could have enforced requirement CR3.4 further by using other methods to scrutinise user input. For example we could have used the Flask escape function to sanitise user inputs in the Python routing files for redundancy.

Testing

Front-end user testing was an integral part of our development process, and we are pleased to report that the overall feedback has been positive. Throughout development, several issues were identified and resolved to improve the user experience on the website. One of the issues addressed was the status and progress colours, which were not matching, causing confusion, and thus by creating more consistency with the colour scheme, we could improve the overall experience and clarity. Another issue that was identified was the sections on the page being displayed all at once, thus having to navigate through them to find important information. The issue was resolved by adding minimizable accordions that allow users to expand and collapse the sections they want to view, thus reducing page clutter. In addition, users provided feedback on the grouping of buttons on the website, which were not always in an accessible location. This feedback led to a reorganisation of the buttons, grouping them based on the actions they perform and placing them in a more intuitive location, thus adhering to the proximity principle, which states that related items should be grouped together to help users understand their relationship and minimise visual clutter.

Before finishing the project, we completed another round of user testing, to get a better idea of how the website is used and the opinions of those using it. In the end, users found the website easy to navigate, with plenty of intuitive features that allowed them to accomplish tasks efficiently. The ordering feature for projects was particularly highlighted as it was simple and user-friendly. The colour scheme used throughout the website was consistent and helpful, making it easier for users to identify different sections and functions. The animations and modals used for entering information were appealing and easy to use/understand, making the overall experience more enjoyable. Consistency with buttons and their animations was appreciated by users, and it was noted that information was well-displayed and well-spaced out, thus eliminating clutter. However, there were some cons that came up during testing. For example, some users found the help button confusing, especially since it was different on the Dashboard and Project Page. Also, there was no way to see email/details about coworkers in the navigation bar, which some users found to be a minor inconvenience. Finally, some minor changes were suggested, such as the online pill in the navigation bar, which looked like an on/off switch and caused some confusion, as well as how the date format was unexpected and should have been day-month-year instead of the current format. Overall, user testing was an essential part of the website development process and helped us to refine the user experience to make it more intuitive and user-friendly.

5 ii Database

Throughout the course of this project, the demands needed from the database changed regularly, and thus, the methods as well. Since the database program includes a plethora of methods, a comprehensive automated unit test program was created to ensure that each method was working according to design and that each update to the database was as intended. After the initial static testing where we inspect code, the test program would be run at regular intervals throughout the sprint to make sure that any new commits to the repository would still meet our requirements. The screenshot in Figure 11 shown below is an example of said test program. For example, we check to ensure that the correct data types are accepted, incorrect types are rejected, and values outside the expected range are rejected as well. The tests also check for integrity constraints (especially referential integrity) and that any inputs and updates by the user on the site are reflected appropriately on the database.

```
-----
SoftENG DynamoDB Test Log
-----
Testing if add functions are working
-----
Added ManagerID 1 to manager Table. Testing if ManagerID 1 exists. Expected: True | Got: True
Added ProjectID 1 to Project Table. Testing if projectID 1 exists. Expected: True | Got: True
Added employeeID 27396098-1662-4875-b2c7-4433c0913114 to EmployeeTable. Testing if EmployeeID exists. Expected: True | Got: True
Added RequirementID 1 to Requirement Table. Testing if Requirement 1 exists. Expected: True | Got: True
Added 4 sprints for ProjectID 1 to Sprints Table. Testing if all sprints are found. Expected: True | Got: True
```

Figure 11: Database Test Program

Other tests that were unfeasible to test using the test program was performed and recorded on a test log, of which a snippet is shown below in Figure 12. A majority of the tests are performed visually on the website to ensure that the website is correctly reflecting data on the database and vice versa.

Step #	Step Details	Expected Results	Actual Results	Pass / Fail
1	Test editing permissions for team members	Team member's permissions should be correctly edited, and the project page should reflect said permissions	As Expected	Pass
2	Check graph data	Graph data should be displayed correctly, and updated when requirements are added/checked off	As Expected	Pass
3	Testing referential integrity (Managers and Employees)	Adding a manager with an existing EmployeeID should be allowed. An error should pop up if an attempt is made to add a manager with no legal EmployeeID	As Expected	Pass

Figure 12: Database Test Logs

For requirement DR1.8, although we did keep track of a project's team members and their skillset, we have not implemented the feature to track what each team member's tasks are, or grouping them by the skills either. A solution would be to add a new prompt for requirements to select what requirements need to be done by each employee. As for the grouping by experience and expertise, a solution would be to add more details such as years of experience when adding team members, and potentially also a drop down list/check box of pre-set skills to easily group employees, as well as potentially suggesting team members for a requirement.

With more time, we might have avoided the sunk cost fallacy which was our use of DynamoDB. Amazon does offer other database services which work with MySQL and PostgreSQL, such as Amazon Relational Database System (RDS), albeit RDS being much more complicated to use and setup compared to DynamoDB. Since most of us are more familiar with SQL as opposed to NoSQL as well as how our tables all reference each other, this change would definitely be the first thing we do if we were to carry out the project again.

5 iii Back End

The `testing_risk_calculator.py` file displays the unit tests for each function in risk calculation including some larger scenario based tests which allow the whole modelled to be evaluated.

```
## TEST for empty exceptions
rc = risk_calculator()
assert rc.requirements_to_overall_severity([]) == 0
assert rc.schedule_metric([]) == 0
assert rc.cost_metric(0,0) == 0
assert rc.quality_metric([]) == 0
```

Figure 13: Simple Unit Tests

Figure 13 shows some of the basic unit tests of the risk calculation functions. In particular, it checks whether they can all handle being called with empty objects as input. Each of the functions seen here also has its own unit tests which checks that it behaves as expected on more sensible input. For example, if we call `quality_metric` with a list of 5's then we should get 0 back as this indicates all of the work being done is of the highest possible standard and is therefore not contributing to the project risk.

```
##Scenario 3: General high risks
req_val = rc.requirements_to_overall_severity([(80, 90), (60, 100), (70, 90), (60, 80)])
schedule_val = rc.schedule_metric([0.2, 0.1, 0.05])
cost_val = rc.cost_metric(10000, 7000)
quality_val = rc.quality_metric([2, 1, 1])
assert rc.risk_metric(schedule_val, req_val, quality_val, cost_val, w) > 4
```

Figure 14: Scenario Based Tests

Figure 14 shows one of the tests for the overall risk model. In this scenario all of the values for each metric are quite high and so we would expect the overall risk value to be high also. There are equivalent scenarios for low risks and also more nuanced scenarios where only one of the individual factors is particularly alarming. These tests show that we have successfully accomplished DR2.1, arguably the most critical back end requirement, as we can produce accurate ratings about how risky a project's current situation is.

Attributes Used	Number (/150)	Percentage Correct
Probability, Priority	11	92.67
Probability, Magnitude, Priority	12	92
Impact, Magnitude	77	48.67
Probability	22	85.33
Priority	8	95.33

Whilst our risk model performs suitably well in testing, various aspects of the model could be improved. The table above shows the performance of various Gaussian Naïve Bayes Models on the requirements dataset using a 50 / 50 train, test split. We see that the pair of attributes used in our model (probability, priority) does not in fact perform the best out of all options. In fact, a simple linear mapping of the priority attribute which is measured 0 to 100 onto the [1, 5] interval (i.e. 0 to 20 maps to 1, 20 to 40 maps to 2 and so on) only mislabels 3 / 150 requirements. This shows the importance of priority as an attribute to track in risk management however, our decision to also track probability is justified by the fact that in other environments it is highly likely that the risk level of a requirements depends on more than one factor and as such should be modelled in a multidimensional way. When the risk model is deployed on customer systems it would be sensible to utilise pre-existing project

data for that customer in order to make sure the model maps as accurately as possible and that the correct attributes are chosen. With more time, more extensive data collection could be done in order to build a better more generalised model.

5 iv System Evaluation

The final product that was achieved was overall a success, despite some of its limitations. Our system, even as a prototype, delivers on the requirements that were set all the way at the very beginning of the project. We have provided a system which lets users track software development projects, and provides them useful and informative feedback. Furthermore, many components are built to be readily expanded upon, with accessible and well documented code. As demonstrated in the system overview, much of the requirements outlined in our specification have been met or changed for greater success. Those requirements that were left incomplete were primarily of low priority, and had little impact on the overall quality of the product.

We may find further success, with special regard to the display, and the research that was conducted. The website is excellently designed to not only provide a comfortable and interactive experience with the user, but also demonstrates a level of professionalism in construction and execution. With the regular user testing to ensure that the most recently added features are intuitive and comfortable to use, both the dashboard and the project page allow for easy interpretation of the data shown. The dashboard uses simplicity to its advantage, with the important data of each project - name, progress, status, deadline - all being displayed clearly to the user. Additionally, the quality of life features of the sorting functionality of projects, and the useful frequently used buttons conveniently placed along the left hand side, lets the user be able to effortlessly navigate the page. It is also important to note, that the way in which the left hand margin is designed, is poised to be expanded upon in future updates. Allowing for easy development follows to the project page, where new details related to sprints or the project, can easily be slid into their designated blocks. The graph system as well enables this, as whenever a new metric can be tracked it can easily be added to the drop down options. The page is full of easy to read detail, and when combining that with its potential for expansion, it is incredibly successful. There are still areas to improve upon though. One of which is the potentially difficult to interpret graphical information. The way that we have designed the risk evaluation, has it currently such that the close to 0 a given project is, the more likely it is to be successful; with 5 being the worst case scenario. This means when looking at the graph, the user is looking for a downward trend which isn't the most intuitive. Moreover, as we only describe the data lexically, with the numerical value in brackets, this may be less easy for the user to read and interpret. Providing alternative methods of displaying the data and specific trends may be preferred for future builds.

Another successful area of the project lies in the findings of the risk analysis. While we did not achieve the final goal of all metrics that we wanted to consider, the four that we prioritised to be completed, requirement evaluation, schedule metric, cost metric and quality metric, are well designed with a lot of statistical and research based backing. In particular, the use of the datasets for the schedule metric [7], and the requirement analysis [12] provided us with the opportunity to make use of Python's data analysis libraries of scipy and sklearn to obtain accurate models we can use to interpret schedule and requirements accurately. Furthermore the encapsulation of the risk calculator allows for changes that may come from other sources of data, or new research later on, to be easily implemented, without breaking the overall system, improving maintainability significantly. However, there are some flaws with the method of risk analysis. Firstly, due to limited datasets, there may be false pretences and biases present within the data. For example, the dataset concerning the schedule is all from one group, which may mean the schedule patterns are unique to them, and not an overall trend. Furthermore, the failure to be able to include the redundancy table and the resourcing metric within the allotted time frame is unfortunate, especially considering the high priority of the redundancy table. These two metrics unfortunately struggled from the lack of available hands; with only one person focused entirely on the risk calculator, there was little that could be done to make sure that our resources were not being spread too thin.

The risk calculator's shortcomings are not the only area where the product fails. While on the surface in the most important areas, the design has been highly successful, the real issues lie with the database implementation, and the interaction between components. The database implementation is the weakest part of the project, due to the lack of security enforcement that is provided by a NoSQL database. Not only did it make scraping information from the database tedious and difficult, but it also makes it notably slower, especially when having to update several elements of sprint values. The largest problem is how it makes it very difficult to expand upon in its current state. The database, its functions, and the operations associated are not easily maintainable. This is incredibly dangerous for other teams tackling the project, and is not an easy quick fix. The lack of features such as date and serialised types required the team to have to reinvent the wheel at points, and the absence of foreign key restrictions means the database may be prone to data not reflecting the realities it is trying to represent. When we are claiming to be able to show the user accurate and useful data, where people's livelihoods may be at risk, the danger imposed by potential misrepresentation is far from ideal. Fortunately, due to careful and clever restrictions put in place by the database team, risks have been minimised, and for this current prototype, the database acts appropriately, as demonstrated by the test logs.

Another area that could see substantial improvement, would be within the interactions between the systems of the product. Currently, the functionality of the project is spread across three main files, a risk calculator class (`risk_calculator`), a database class (`softEngTable`) and the front end main file calling interactions between these. However, when looking at the `softEngTable`, we can see that it is divided up into different functionalities, which would have served greatly as their own singleton calls. Especially to distinguish event functions and initialization functions. This lack of utilization of the object oriented design slightly hinders the projects future extensibility, having to work within the current confines of the set class, reducing the modularity as well.

Despite all of those issues, the project is still a success. The particular reason for this is the ability of our risk calculator. Within the files is `test_data.py` which contains several simulations of existing projects. These simulations use the interactivity functions defined in a imperative way to simulate the real world passage of time as if the project is actually occurring (with some small abstractions such as cost being updated all in one go each sprint). With these simulations, each one is designed around being successful or flawed in various ways; though rather than building around the existing algorithms, it is built around general successes and failures. Here we consider roughly less than 1.5 is ahead, between 1.5 and 3.5 is on track, and above 3.5 is at risk. In particular, remember that the closer a project is rated to 0, the better it is. The test log below details what each project was trying to show, and if it was successful. More details about the tests can be found in the program files.

Project	Goal	How	Risk Value	Successful?
Sonic Adventure 3	This is a project that is supposed to be on track.	This was done by balanced requirements, completion of a couple requirements are delayed, and budget is maintained.	2.9955	This is accurate, as between 1.5 and 3.5 projects are on track. Sonic Adventure 3 is within a comfortable range.
Communication encryption system	This is a project that is performing well.	This was done where requirements are receiving high quality reviews, and near most requirements are being completed on time. Budget is being maintained to be on track to spend less than budget, and requirements are set to have a high risk to them.	1.3006	This is very accurate, as while the project has high quality on schedule, the requirements are poor and budget is average, hence it is close to the boundary value of 1.5 but it is clearly ahead.
Tabula Update	This is a project that is supposed to be failing.	This is designed with ambitious requirements, with many requirements not completed within their sprint, and when they are completed they attain low scores.	3.9080	This reflects the truth that the project is failing, however it's value fails to depict how significantly.

5 v Extensions

As a software development tracker, this project has a lot of potential for further extension and improvement.

Visual and interactivity

One way to enhance the user experience would be to introduce more responsiveness to the website. This would allow the website to function with a wider array of screen sizes, from small mobile devices to larger desktop computers. This could be achieved by using responsive design techniques such as fluid grid layouts, flexible images, and media queries that adjust the website's layout and content based on the device's screen size. This would ensure that the website is easily accessible and usable for all users, regardless of their device.

Another way to improve the website's functionality and user experience would be to use JavaScript for updating the page without requiring reloads or redirects. This would provide a more seamless and interactive experience for users, as they would be able to see changes to the website's content and functionality without having to wait for the page to reload. This could include dynamic loading of content, live updating of progress bars or graphs, and the ability to drag and drop elements on the page. Additionally, more animations could be added to elements on the website, such as menu transitions, hover effects, and page transitions. This would add a visually appealing and engaging aspect to the website, making it more enjoyable to use for the users.

To further extend the functionality of the website, one possible improvement would be to allow the navigation bar to show more than just five projects and coworkers. Additionally, it would be beneficial to add more functionality directly to the navigation bar, such as creating a new project or adding new team members, without the need to navigate to specific pages. This would make it easier and more efficient for users to access and manage their projects and team members.

Another potential extension for the website would be to provide more sorting options for users. Currently, users can sort projects based on name, deadline, and budget. However, there may be other attributes that are important to users, such as priority or status, that could be added as sorting options. This would make it easier for users to quickly find the projects they need to work on, and improve the overall user experience.

Finally, an additional extension that could be made is to expand the three levels of permissions to a more fine-grained, custom permission system. This would allow each team member to have personalised access to certain areas of a project based on their role and responsibilities. For example, a designer may need access to different areas of a project than a developer, and this custom permission system would allow for that level of specificity. This would improve the overall security and control of the website, and make it easier for team members to focus on their specific tasks.

Interpretation and metrics

One of the larger areas for development surrounds the consideration and expansion of how we calculate risk. Not only are there potential methods of expanding the current metrics that we consider, but also implementing the ones that we have neglected. Firstly for our existing models for requirements and schedules, we only use the one dataset that we had access to. However, with more time we would be able to reach out and research more data, from academic and corporate sources. Then using this we could either compile it with our existing data, or enable for the user to choose alternative models to pull from, different models being more suited toward certain projects. This would apply to all future metrics utilizing a given dataset. Additionally we would enable managers to determine the weighting of each metric for a given metric, so they can place focus in certain areas. As for considering other metrics, there are a large number of metrics and methods we could use; however if we had more time these are how we would start these measures.

Redundancy table - A project not able to accommodate budget for potential realised risk is one of the largest reasons that most projects fail. This could be implemented in our tracker, by having an option to add risks, their associated cost, and their probability of occurring. Common business practices sees that their budget must account for $\sum C(r) \cdot P(r)$ where $C(r)$ is the cost of realised risk of r , and $P(r)$ is the probability of risk r occurring.

Stakeholder satisfaction - This was originally intended to be included for the prototype, and would allow for the manager to be able to enter in review from their stakeholder (perhaps storing a transcript as well), giving a score. Following this, the metric would track not only the score, but also its trend using the sum of least squares to attain whether the spread of data is trending to improve; which is crucial for ensuring a final product that meets the stakeholder's demands.

Resourcing - This metric would be using a combination of the stakeholder satisfaction metric, the quality metric and the schedule metric to determine the rating of the assignment of a person or a team of people to a task. This would then be accumulated for all teams, and used to evaluate current assignment. This would be difficult to implement, but it would be picking up on patterns of teams producing lower quality work, that is inconsistently completed. Then the system would advise potential reassignments of teams.

Soft metrics - This possible extension would utilize the feedback feature, and take into account details of their mood, satisfaction, and perhaps even interpretation of lexical data. This would be difficult to interpret unless datasets were found; however this data can theoretically be built, as projects develop, the employee's morale and overall productivity can be paired and interpreted. Irrespective, it would be useful to store for the manager's own use, however they see fit.

6 Process Evaluation

6 i Methodology

Pair Programming

At the beginning of development, the individual teams split up according to their workload and focused individually on tasks. As time progressed, the frontend team quickly realised the many repetitions and issues with code they were having were common

between developers. As such, to streamline the process, the frontend team has decided to experiment with pair programming. Pair programming is a technique where two programmers work together on one workstation, sharing a keyboard and a mouse, while one types and the other reviews. To better accommodate ourselves, the developers decided to work on separate computers with a shared screen, as this allowed for the reviewer to have their independence to test separate code or research information while the other programs. As the usefulness was quickly observed, the front-end team of the website implemented pair programming for the majority of the development time, and it has greatly reduced developing time, fostering knowledge sharing, and enhancing problem-solving abilities.

Scrum and Sprints

Throughout the development cycle, our team followed a Scrum agile approach to better manage our workload and ensure that we stay on track to meet our project deadlines. As a part of this approach, we held stand-up meetings twice a week, online on Monday and in-person Friday. On Monday, we discussed specific updates related to the development of the project, while on Friday, we discussed general information related to the entire weekly sprint. These meetings helped us to stay on track with the project's goals, address any issues that might have arisen, and ensure that everyone was aware of what tasks needed to be completed, as each team was given their respective tasks for sprints and had to remain accountable for their completion. To further aid us in keeping track of tasks and ensuring that our development process was as efficient as possible, we used Jira, a software tool designed for agile project management. Jira provided us with a range of tools and features that allowed us to keep track of all our tasks, assign them to specific team members, and track their progress over time, with an example shown in the following image. This helped us to ensure that all tasks were being completed on schedule and that everyone was on the same page when it came to the project's progress. We also made use of GitHub for version control and file sharing throughout the development process by creating branches to work on tasks individually and merging after their completion. Despite some minor hiccups due to the initial learning curve of using git, GitHub proved to be an excellent tool for managing the different versions of our code and ensuring that everyone on the team had access to the latest version. This allowed us to collaborate more effectively, share ideas and insights, and ultimately produce a better final product.

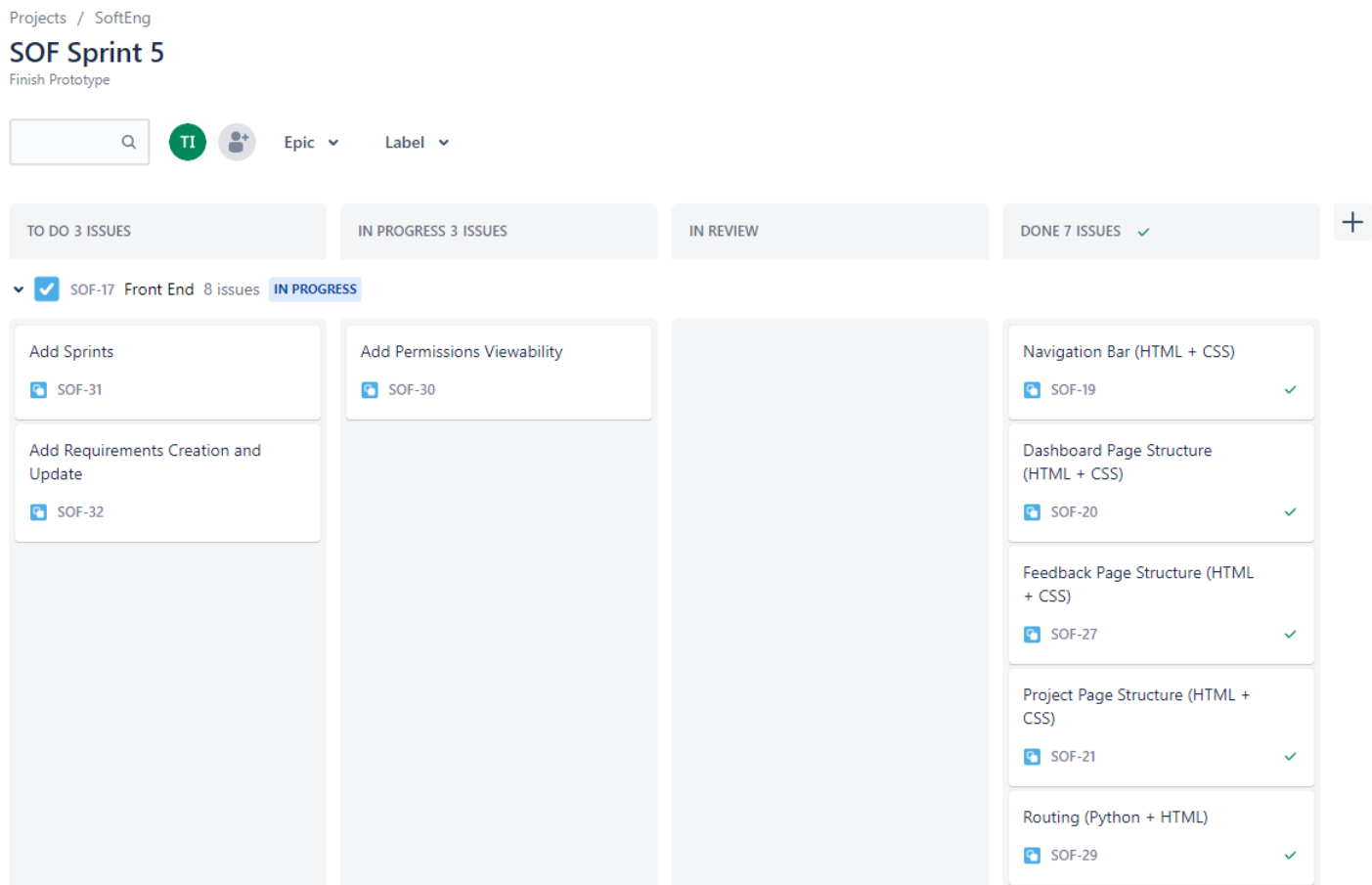


Figure 15: A Sprint Overview in Jira

6 ii Team Management

Our small team of five students as elicited in the design document broke down into smaller subteams which each had their own focus. Young-Ha Chun and Tudor Irimies were the web developers, whereas Yi Xiang Ng and Owen Green were in charge of back end development, while Archie Harrodine remained as a floater between the two. By separating out the team like so, it enabled much faster aforementioned pair programming sessions, and a level of independence to act by one's self. This independence was steered on track by the regular frequent meetings, which meant that focus was maintained, and interactivity could be ensured. This division certainly sped up development, and enabled for well performing individual subsystems. However, this was a doubled edged blade, as the division of labour also meant that there was a natural assumption of a black box that others may call to for what was required, which in the last sprints of the project had serious ramifications. Certain issues with interaction came up, one of which being the issue of the permissions, which reworked the database in a way that was difficult and awkward for others; but due to the independent nature, the difficulty was not found until it was too late to change. While the issues that arose were worked around, it did sap time and energy from all team members involved.

The structure of division could have worked far better with some improvements. The chief improvement it could have used was that each work session, members of each team list what is needed from the other side. For example, detailing function inputs and outputs well in advance, expressing need of a certain input accessed from the other team. This would have meant that issues of interaction could have been faced far earlier on, and implementation could have been tackled more smoothly. Another change that would have been beneficial, would have been for more sessions to explain intention and design between team members. There were several occasions throughout the project where progress was stunted due to one team not fully understanding what was wanted of the other. This was most apparent when the web developer team were not fully understanding of the new AGILE focus and method of how that would be conducted. Although more sessions would have been ideal, that neglects the reality of that each of us had busy university schedules that collided with the project, making more frequent sessions difficult to achieve.

The methodologies practiced of pair programming and the overall scrum approach were highly beneficial toward the success of the project. Likewise the team division enabled speed in development that otherwise could not have been achieved. With some changes to how the methodologies were put in practise, the project's development could have been improved, for a more seamless experience. Despite the issues though, the team worked together cohesively, and produced a successful working product.

7 Summary

In general, the report highlights the success of the prototype that effectively tracks software development projects and provides valuable feedback to its users. The website is designed to not only provide a comfortable and interactive experience with the user, but also demonstrates a level of professionalism in construction and execution, facilitating the user in keeping track of projects they are part of and providing an enjoyable experience when working. Through quality of life features such as the sorting functionality of projects and the useful frequently used buttons conveniently placed along the left hand side, the user is able to effortlessly navigate the pages and achieve their tasks.

The risk analysis findings in the project are well-designed, backed by statistical and research-based data. The use of datasets and data analysis libraries allowed accurate interpretation of the four metrics, namely requirement, schedule, quality and cost values, while the encapsulation of the risk calculator enabled a high degree of modularity and thus facilitating implementation of changes in the future, improving maintainability significantly. Despite some flaws with the method of risk analysis due to limited datasets, the project delivered on the prioritised metrics, providing a strong foundation for further improvements in future versions.

Overall, besides some areas of the project that require improvement, such as the database implementation and the interaction between components, the design has been highly successful in important areas. The team has taken careful and clever measures to minimise risks, and the database team has done a good job making sure the database works appropriately for the current prototype, as demonstrated by the test logs. While the lack of relationship enforcement provided by a NoSQL database has made scraping information from the database tedious and difficult, and it has made it slower, the team has been able to work around these limitations to show the user accurate and useful data. While further improvements can be made, the current prototype is a success in important areas.

References

- [1] Using the Central Limit Theorem, November 2021. [https://stats.libretexts.org/Bookshelves/Introductory_Statistics/Book%3A_Introductory_Statistics_\(OpenStax\)/07%3A_The_Central_Limit_Theorem/7.04%3A_Using_the_Central_Limit_Theorem](https://stats.libretexts.org/Bookshelves/Introductory_Statistics/Book%3A_Introductory_Statistics_(OpenStax)/07%3A_The_Central_Limit_Theorem/7.04%3A_Using_the_Central_Limit_Theorem) Last Accessed: 12th March 2023.
- [2] Fernando Acebes, María Pereda, David Poza, Javier Pajares, and José Manuel Galán. Stochastic earned value analysis using monte carlo simulation and statistical learning techniques. *International Journal of Project Management*, 33(7):1597–1609, 2015.
- [3] Beatrice Akumba, Samera Otor, Iorshase Agaji, and Barnabas Akumba. A predictive risk model for software projects' requirement gathering phase. *International Journal of Innovative Science and Research Technology*, 5:231–236, 2020.
- [4] Michael Bloch, Sven Blumberg, and Jürgen Laartz. Delivering large-scale it projects on time, on budget, and on value, October 2012. <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/delivering-large-scale-it-projects-on-time-on-budget-and-on-value/> Last Accessed: 12th March 2023.
- [5] Dale Healey. Understanding the cost performance index (cpi) on your project, 2020. <https://logikalprojects.com/insights/understanding-the-cost-performance-index-cpi-on-your-project/> Last Accessed: 12th March 2023.
- [6] Byung-Cheol Kim and Jeffrey K. Pinto. What cpi = 0.85 really means: A probabilistic extension of the estimate at completion. *Journal of Management in Engineering*, 35(2):04018059, 2019.
- [7] Randula Koralage. Agile scrum sprint velocity dataset, August 2020. <https://github.com/RandulaKoralage/AgileScrumSprintVelocityDataSet> Last Accessed: 12th March 2023.
- [8] Jan Kubovy, Christian Huber, Markus Jäger, and Josef Küng. A secure token-based communication for authentication and authorization servers. pages 237–250, 11 2016.
- [9] Leila Moslemi Naeni and Amir Salehipour. Evaluating fuzzy earned value indices and estimates by applying alpha cuts. *Expert Systems with Applications*, 38(7):8193–8198, 2011.
- [10] The pandas development team. pandas-dev/pandas: Pandas, February 2023. <https://doi.org/10.5281/zenodo.7658911> Last Accessed: 12th March 2023.
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [12] Zain Shaukat, Rashid Naseem, and Muhammad Zubair. A dataset for software requirements risk prediction. In *2018 IEEE International Conference on Computational Science and Engineering (CSE)*, pages 112–118, 2018.
- [13] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.